



**Khawaja Fareed University of
Engineering & Information Technology**

YOLO Pose Detection Web Application Using Django

A Web-Based Computer Vision Project

Project Title:

YOLO-Based Human Pose Detection Web Application

Submitted By:

Abdul Hanan

Roll Number:

Dsai-232103032

Submitted To:

Mr. Muhammad Waleed

Department:

AINT-6A

Department of Data Science & Artificial Intelligence

YOLO Pose Detection Web Application (Django)

1. Introduction

This project is a **YOLO-based Pose Detection Web Application** developed using **Django**. The main purpose of this system is to perform **human pose detection** on images/video streams using a **pre-trained YOLO Pose model (.pt file)** and display the results on a webbased frontend.

⚠ **Note:** Model training is **NOT** part of this project. The model was trained beforehand, and only the trained `.pt` file is used for inference in Django.

2. Project Objectives

- Integrate a pre-trained YOLO Pose Detection model into Django
 - Provide a user-friendly web interface (frontend)
 - Perform pose detection on images/video using backend processing
 - Display detected keypoints and skeleton on the frontend
 - Store processed images (optional) in the database
-

3. Technologies Used

Project Structure Overview

The project follows a clean and modular Django structure to separate frontend, backend, and machine learning components clearly.

```
CV_PROJECT/
├── env/                    # Python virtual environment |
├── myproject/
│   ├── media/             # Stores output images/videos after pose
│   ├── detection
│   │   ├── model/         # Contains pre-trained YOLO pose model (.pt
│   │   │   │               file)
│   │   ├── models/        # Optional ML/helper modules |
│   │   └── myapp/          # Main Django application
│   └── migrations/        # Database migration files
```

```

├── templates/
│   └── dji_pose_detection.html # Frontend UI
├── views.py # YOLO pose detection backend logic
├── utils.py # Helper functions (image & skeleton drawing)
├── urls.py # URL routing
├── models.py # Database models (optional)
├── admin.py # Admin panel configuration
├── apps.py # App configuration
├── myproject/ # Django settings folder
├── db.sqlite3 # SQLite database
├── manage.py # Django management script
└── requirements.txt # Project dependencies

```

This structure helps in maintaining scalability, readability, and easy deployment of the application.

4. Technologies Used

Backend

- Python
- Django
- Ultralytics YOLO
- OpenCV (cv2)
- NumPy
- Pillow (PIL)

Frontend

- HTML
- CSS
- Bootstrap
- JavaScript

Model

- YOLO Pose Detection model (.pt file – pre-trained)

4. System Architecture

The system follows a **Client–Server Architecture**:

1. **User (Client)** uploads an image or starts camera feed
2. **Frontend** sends data to Django backend

3. **Backend** loads YOLO .pt model
 4. Pose detection is performed
 5. Processed output (keypoints/skeleton) is returned
 6. **Frontend** displays the results
-

5. Project Workflow

1. User opens the web application
 2. User uploads an image or enables camera
 3. Django receives input via view
 4. YOLO model processes the image
 5. Pose keypoints & skeleton are detected
 6. Output image is generated
 7. Result is displayed on the webpage
-

6. Model Integration (Without Training)

- The YOLO Pose model is already trained
- The .pt file is placed inside the Django project directory □ The model is loaded using the **Ultralytics YOLO API**

```
from ultralytics import YOLO model
= YOLO('yolo_pose.pt')
```

- The model is used only for **inference (prediction)**
 - No training or fine-tuning is performed in this project
-

7. Backend Implementation

Views

- Handle image/video input
- Load YOLO model
- Perform pose detection
- Convert output to displayable format

Image Processing

- OpenCV is used for:
 - Image reading
 - Drawing keypoints

Drawing skeleton
lines

Database (Optional)

- Detected images can be stored using Django models
 - Uses `ImageField` for saving output images
-

8. Frontend Implementation

Features

- Camera start/stop
- Image upload option
- Real-time pose visualization
- Display keypoints and skeleton

UI Components

- Video container
 - Canvas for drawing pose
 - Buttons for user interaction
-

9. Key Functionalities

- Human pose detection
 - Keypoints detection (head, arms, legs, etc.)
 - Skeleton visualization
 - Real-time or image-based detection
 - Web-based interface
-

10. Advantages of the System

- No need for model training
 - Easy deployment
 - Fast inference
 - User-friendly interface
 - Scalable architecture
-

11. Limitations

- Model accuracy depends on pre-trained weights
- Requires good lighting for better results
- Performance depends on system hardware

12. Future Enhancements

- Add video file upload support
- Improve UI/UX
- Add multi-person tracking
- Optimize performance using GPU
- Export pose data in JSON format

13. Conclusion

This Django-based YOLO Pose Detection project successfully demonstrates how a **pretrained deep learning model** can be integrated into a **full-stack web application**. The system focuses on **model deployment and inference**, not training, making it suitable for real-world applications such as fitness tracking, surveillance, and posture analysis.

14. References

- Ultralytics YOLO Documentation
- Django Official Documentation
- OpenCV Documentation